

Exercício: Biblioteca no Boilerplate (Meu Acervo e Meus Empréstimos)

Turma: LionsDev

Tópicos: Boilerplate, rotas protegidas com JWT, dono do recurso via token, dois recursos, relacionamento com `ObjectId` / `ref`, regra que cruza coleções, controle de estoque, validações e status codes.

1. Contexto

No Módulo 08 você fez a API da Biblioteca Lions com dois recursos (Material e Empréstimo) no `server.js`. Agora ela vai para o boilerplate em camadas, com uma novidade: cada bibliotecário tem o próprio acervo. Materiais e empréstimos pertencem ao usuário logado.

Nível: avançado. São dois recursos, um relacionamento por `ObjectId` e uma regra que cruza coleções. Faça os exercícios de Petshop e Academia antes.

2. Ponto de Partida: o Boilerplate

Partimos do boilerplate LionsDev: <https://github.com/nicolassmotta/boilerplate-lions-dev.git>

- Clone, `npm install`, crie o `.env` (`MONGO_URI`, `JWT_SECRET`, `JWT_EXPIRES_IN`, `BCRYPT_SALT_ROUNDS`) e suba o servidor.
- Já estão prontos: camada de `Usuario`, cadastro/login com `bcrypt/JWT`, middleware `autenticar` (preenche `req.usuario = { id, email }`), `criarErro` e o middleware de erro.

Antes de começar, pegue seu token e envie `Authorization: Bearer SEU_TOKEN` em todas as rotas novas.

3. Regras do Sistema (valem para os dois recursos)

- Toda rota é protegida.** `router.use(autenticar)` no topo de cada arquivo de rotas.
- O dono vem do token** (`req.usuario.id`), nunca do body.
- Toda consulta filtra pelo dono**, tanto em Material quanto em Empréstimo.
- Filtre pelo dono já na consulta.** Se não achar → `404`, sem `if` de autorização.
- Listagem nunca dá 404:** sem registros, devolva array vazio.

4. Arquivos que você vai criar

Camada	Material	Empréstimo
Model	<code>src/models/material.model.js</code>	<code>src/models/emprestimo.model.js</code>

Camada	Material	Empréstimo
Repository	<code>src/repositories/material.repository.js</code>	<code>src/repositories/emprestimo.repository.js</code>
Service	<code>src/services/material.service.js</code>	<code>src/services/emprestimo.service.js</code>
Controller	<code>src/controllers/material.controller.js</code>	<code>src/controllers/emprestimo.controller.js</code>
Routes	<code>src/routes/material.routes.js</code>	<code>src/routes/emprestimo.routes.js</code>

5. Models

5.1 Material

- `titulo` : `String` , obrigatório, `trim` .
- `tipo` : `String` , obrigatório, `enum`: ["Livro", "Revista", "Apostila"] .
- `autor` : `String` , obrigatório, `trim` .
- `estoque` : `Number` , obrigatório, `min`: [0, "0 estoque não pode ser negativo."] .
- `usuario` : `ObjectId` , `ref`: "Usuario" , obrigatório.
- `timestamps`: `true` .

5.2 Emprestimo

- `material` : `ObjectId` , `ref`: "Material" , obrigatório (qual material foi emprestado).
- `nomeAluno` : `String` , obrigatório, `trim` .
- `turma` : `String` , obrigatório.
- `dataEmprestimo` : `String` , obrigatório.
- `diasEmprestimo` : `Number` , obrigatório, `min`: 1 .
- `multaPrevista` : `Number` (a API calcula).
- `status` : `String` , `enum`: ["Emprestado", "Devolvido", "Atrasado"] , `default`: "Emprestado" .
- `usuario` : `ObjectId` , `ref`: "Usuario" , obrigatório (o dono do empréstimo).
- `timestamps`: `true` .

Critérios de aceite: os campos obrigatórios e os `enum` são respeitados; `estoque` não aceita valor negativo.

Conceito novo: relacionar dois recursos com `ObjectId` + `ref`

Como nos outros exercícios, `usuario` é o dono. A novidade é que o `Emprestimo` tem mais uma referência: ele aponta para o `Material` emprestado, guardando o `_id` dele.

```
import mongoose from "mongoose";
// emprestimo.model.js
material: { type: mongoose.Schema.Types.ObjectId, ref: "Material", required: true },
usuario: { type: mongoose.Schema.Types.ObjectId, ref: "Usuario", required: true },
```

Guardando o `_id` , mais tarde dá pra usar `.populate("material")` para trazer os dados completos do material junto (veja o bônus).

6. Repositories (sempre filtrando por dono)

6.1 MaterialRepository

- `criar(dados) → Material.create(dados) .`
- `listarPorUsuario(idUsuario) → Material.find({ usuario: idUsuario }).sort({ createdAt: -1 }) .`
- `listarDisponiveisPorUsuario(idUsuario) → Material.find({ usuario: idUsuario, estoque: { $gt: 0 } }) .`
- `buscarPorIdDoDono(idMaterial, idUsuario) → Material.findOne({ _id: idMaterial, usuario: idUsuario }) .`
- `salvar(material) → material.save()` (útil para atualizar o estoque).

6.2 EmprestimoRepository

- `criar(dados) → Emprestimo.create(dados) .`
- `listarPorUsuario(idUsuario) → Emprestimo.find({ usuario: idUsuario }).sort({ createdAt: -1 }) .`
- `buscarPorIdDoDono(idEmprestimo, idUsuario) → Emprestimo.findOne({ _id: idEmprestimo, usuario: idUsuario }) .`
- `salvar(emprestimo) → emprestimo.save()` .
- `deletarPorIdDoDono(idEmprestimo, idUsuario) → Emprestimo.findOneAndDelete({ _id: idEmprestimo, usuario: idUsuario }) .`

Dica: como o empréstimo precisa alterar o estoque do material, é mais fácil buscar o documento, mudar um campo e chamar `.save()` do que usar `findOneAndUpdate` . Por isso há um `salvar(...)` nos dois repositories.

Conceito novo: `.save()` para atualizar (ler, mudar, gravar)

Quando a nova informação depende da atual (estoque atual - 1), busque o documento, altere em memória e grave:

```
const material = await MaterialRepository.buscarPorIdDoDono(idMaterial, idUsuario);
material.estoque = material.estoque - 1; // muda em memória
await material.save(); // grava no banco
```

O `salvar(doc)` faz só isso: `doc.save()` .

Conceito novo: operadores de consulta (`$gt`)

Para listar só o que tem "estoque maior que zero", o filtro usa um operador do MongoDB:

```
Material.find({ usuario: idUsuario, estoque: { $gt: 0 } });
```

`$gt` = *greater than* (maior que). Existem também `$gte` ($\geq$), `$lt` ($<$) e `$lte` ($\leq$).

7. Services (regras de negócio)

7.1 MaterialService

- `registrar(idDoUsuario, dados)` : monta o objeto com `usuario: idDoUsuario` e cria.
- `listarMeus(idDoUsuario)` : lista os materiais do usuário.
- `listarDisponiveis(idDoUsuario)` : lista só os com `estoque > 0`.

7.2 EmprestimoService : a parte principal do exercício

`registrar(idDoUsuario, dados)` :

1. **O material é seu e existe?** Busque com `MaterialRepository.buscarPorIdDoDono(dados.material, idDoUsuario)`. Se vier `null` → `criarErro("Material não encontrado.", 404)`.
2. **Tem estoque?** Se `material.estoque <= 0` → `criarErro("Material sem exemplares disponíveis.", 400)`.
3. **Multa prevista:** até 7 dias sem multa. Acima disso, R\$ 2 por dia extra (`(diasEmprestimo - 7) * 2`).
4. **Baixa no estoque:** `material.estoque = material.estoque - 1` e `MaterialRepository.salvar(material)`.
5. **Cria o empréstimo** com `usuario: idDoUsuario`, o `material`, a `multaPrevista` calculada e o restante dos dados.

`listarMeus(idDoUsuario)` : retorna a lista do usuário.

`buscarMeu(idDoUsuario, idEmprestimo)` : `buscarPorIdDoDono(...)` ; se `null` → `404`.

`alterarStatus(idDoUsuario, idEmprestimo, novoStatus)` :

1. Busque o empréstimo pelo dono. Se `null` → `404`.
2. **Regra de estoque:** se o `novoStatus` for `"Devolvido"` e o status atual ainda não for `"Devolvido"`, devolva 1 ao `estoque` do material correspondente (busque o material pelo dono e `salvar`).
3. Atualize o `status` do empréstimo e salve.

`removerMeu(idDoUsuario, idEmprestimo)` : `deletarPorIdDoDono(...)` ; se `null` → `404` ; se ok → `{ message: "Empréstimo removido com sucesso." }`.

Critérios de aceite: emprestar baixa 1 do estoque; emprestar material zerado → `400` ; emprestar material de outro usuário → `404` ; devolver soma 1 de volta ao estoque (uma única vez).

8. Controllers e Routes

Crie os controllers seguindo o mesmo padrão dos outros exercícios (`try/catch` + `next(error)`), lendo sempre `req.usuario.id` , `req.params.id` e `req.body` .

`material.routes.js` (`router.use(authenticar)` no topo):

- `POST /` → registrar · `GET /` → listar meus · `GET /disponiveis` → listar disponíveis.

Cuidado com a ordem: declare `GET /disponiveis` antes de qualquer `GET /:id` , senão o Express trata `"disponiveis"` como um `:id` .

`emprestimo.routes.js` (`router.use(authenticar)` no topo):

- `POST /` → registrar · `GET /` → listar meus · `GET /:id` → buscar meu · `PATCH /:id/status` → alterar status · `DELETE /:id` → remover meu.

No `src/app.js` , antes do middleware 404:

```
import materialRoutes from "../routes/material.routes.js";
import emprestimoRoutes from "../routes/emprestimo.routes.js";

app.use("/api/materiais", materialRoutes);
app.use("/api/emprestimos", emprestimoRoutes);
```

9. Rotas finais

Método	Rota	Proteção	Descrição
POST	<code>/api/materiais</code>	JWT	Cadastrar material no meu acervo
GET	<code>/api/materiais</code>	JWT	Listar meus materiais
GET	<code>/api/materiais/disponiveis</code>	JWT	Listar meus materiais com estoque
POST	<code>/api/emprestimos</code>	JWT	Registrar empréstimo (baixa estoque)
GET	<code>/api/emprestimos</code>	JWT	Listar meus empréstimos
GET	<code>/api/emprestimos/:id</code>	JWT	Ver um empréstimo meu
PATCH	<code>/api/emprestimos/:id/status</code>	JWT	Mudar status (devolução soma estoque)
DELETE	<code>/api/emprestimos/:id</code>	JWT	Remover um empréstimo meu

10. Fluxo de Testes (use Postman)

1. Cadastro/login → copie o `token` .
2. `POST /api/materiais` com `estoque: 1` → guarde o `_id` do material.
3. `POST /api/emprestimos` com `material` = esse `_id` e `diasEmprestimo: 10` → `201` , `multaPrevista` = 6.
4. `GET /api/materiais/:id` (bônus) ou liste e confira que o `estoque` virou `0` .

5. **POST /api/emprestimos** de novo no mesmo material → **400** (sem estoque).
6. **PATCH /api/emprestimos/:id/status** com **{ "status": "Devolvido" }** → **200** ; confira o estoque voltar para **1** .
7. Com um segundo usuário, tente emprestar usando o **_id** do material do primeiro → **404** .
8. Qualquer rota sem token → **401** .

11. Desafios Bônus

1. **GET /api/emprestimos/relatorio** : em JavaScript, retorne total de empréstimos seus, soma das **multaPrevista** e a contagem por **status** .
2. Use **.populate("material")** ao listar empréstimos para trazer os dados do material junto:

```
Emprestimo.find({ usuario: idUsuario }).populate("material");
```

Sem **populate** , o campo **material** vem só com o **_id** ; com ele, vem o objeto inteiro do material.

3. Bloqueie um novo empréstimo se o mesmo **nomeAluno** já tiver um empréstimo **"Emprestado"** seu do mesmo material.